

Leetcode - NTB

• Easy → #1 → Two Sum

▲ **Problem Statement** → Given an array "nums" & an integer "target", return the indices of two different numbers whose sum equals "target".

Ex: nums = [2, 7, 11, 15]
target = 9

Now: $\text{nums}[0] + \text{nums}[1] = 2 + 7 = 9$

So, Answer = [0, 1]

▲ **Main Problem Solution** → We use an "unordered_map" to store each number & its index. For every number, calculate :-

$$\text{required} = \text{target} - \text{nums}[i]$$

If "required" exists in the map, return both indices. Otherwise, store the current number & its index.

▲ **Brute Force** → Use two loops & check every pair :-
 $\text{nums}[i] + \text{nums}[j] == \text{target}$

If true, return indices.

▲ **Algorithm** → 1] Create an "unordered_map".

2] Loop through "nums"

3] Calculate "required = target - nums[i]"

4 | If "required" exists in the map, return its index & "i".

5 | Otherwise, store "nums[i]" & "i"

▲ Important Points → Return Indices, not Numbers

• "unordered_map" stores number → ~~index~~ index

• Check before storing the current number.

▲ Complexity → Brute Force → $O(n^2)$ & $O(1)$
Optimized → $O(n)$ & $O(n)$

★ Easy → #9 → Palindrome Number

▲ Problem Statement → Given an integer "x", return "true" if it is a palindrome, otherwise return "false"